

# CktFormalizer: Autoformalization of Natural Language into Circuit Representations

Jing Xiong\*, Qi Han, Chenchen Ding, He Xiao,  
Zunhai Su, Xiachong Feng, Chaofan Tao, Ngai Wong  
The University of Hong Kong

## Abstract

LLMs can generate hardware descriptions from natural language specifications, but the resulting Verilog often contains width mismatches, combinational loops, and incomplete case logic that pass syntax checks yet fail in synthesis or silicon. We present CKTFORMALIZER, a framework that redirects LLM-driven hardware generation through a dependently-typed HDL embedded in Lean 4. Lean serves three roles: (i) *type checker*: dependent types encode bit-width constraints, case coverage, and acyclicity, turning hardware defects into compile-time errors that guide iterative repair; (ii) *correctness firewall*: compiled designs are structurally free of defects that cause silent backend failures (the baseline loses 20% of correct designs during synthesis and routing; CKTFORMALIZER preserves all of them); (iii) *proof assistant*: the agent constructs automated theorem equivalence proofs over arbitrary input sequences and parameterized widths, beyond the reach of bounded SMT-based checking. On VerilogEval (156 problems), RTLLM (50 problems), and ResBench (56 problems), CKTFORMALIZER achieves simulation pass rates competitive with direct Verilog generation while delivering substantially higher backend realizability: 95–100% of compiled designs complete the full synthesis, place-and-route, DRC, and LVS flow. A closed-loop PPA optimization stage yields up to 35% area reduction and 30% power reduction through validated architecture exploration, with automated theorem proofs ensuring that each optimized variant remains functionally equivalent to its formal specification.

## 1 Introduction

Hardware design remains one of the most error-prone and labor-intensive disciplines in computing. A single bit-width mismatch or missing case in a finite state machine can propagate silently through synthesis, only to surface weeks later as a silicon bug costing millions of dollars to fix. Despite decades of progress in electronic design automation (EDA), the dominant hardware description language (HDL), Verilog and VHDL, still lack the static guarantees taken for granted in software: dependent types, exhaustive pattern matching, and machine-checked proofs of correctness.

Two largely independent research threads have sought to address this gap. On one hand, *type-safe HDL* such as Chisel [Bachrach et al., 2012], Bluespec [Nikhil, 2004], and Clash [Baaij et al., 2010] embed hardware descriptions in host languages with richer type systems, eliminating entire classes of bugs at compile time. On the other hand, *large language models* (LLMs) demonstrate remarkable ability to generate Verilog from natural language specifications [Thakur et al., 2023, Liu et al., 2023], promising to democratize hardware design by lowering the barrier to entry. Yet these two threads remain disconnected: LLM-generated Verilog inherits the pitfalls of the target language

\*Corresponding author: junexiong@connect.hku.hk

and the ambiguity of natural language, while type-safe HDLs still require expert knowledge to use effectively. Table 1 summarizes the key differences. We argue that these threads are *complementary*: an HDL with dependent types provides precise, formal constraints that enable an LLM agent to reason about functional equivalence and make steady progress, particularly in settings where *SMT-based* (Satisfiability Modulo Theories) verification often struggles to converge.

In this paper, we present CKTFORMALIZER, a framework that unifies LLM-driven code generation with formal verification by targeting a dependently-typed hardware domain-specific language (DSL) embedded in Lean [Moura and Ullrich, 2021]. Given a natural language specification, an LLM agent generates hardware whose bit-width safety, loop freedom, and case exhaustiveness are enforced at compile time. The compiled design is automatically extracted to synthesizable SystemVerilog, simulated against reference testbenches, and physically synthesized via OpenROAD [Ajayi et al., 2019], an open-source digital layout implementation toolchain, with the SkyWater 130nm PDK [Google and SkyWater Technology, 2020]. A closed-loop PPA optimization stage feeds synthesis metrics back to the agent for iterative refinement with equivalence guarantees.

The key insight behind CKTFORMALIZER is that *the compiler is the verifier*. Rather than generating Verilog and hoping it is correct, the agent receives immediate, actionable feedback from Lean’s type system at every iteration, feedback that is far more informative than the cryptic warnings of a Verilog linter. This transforms the LLM’s generate-and-test loop from a stochastic search over syntactically valid programs into a *type-guided refinement* process that systematically narrows the space of candidate designs. Our contributions are as follows:

- We introduce CKTFORMALIZER, the first framework combining LLM-driven hardware generation with a Lean-embedded HDL, providing an end-to-end pipeline from natural language to synthesizable silicon with closed-loop PPA optimization.
- We demonstrate that targeting a type-safe HDL substantially improves LLM success rates compared to direct Verilog generation, as compile-time type errors provide structured feedback that guides the agent toward correct designs. The framework further scales to industry-relevant IP such as multi-channel handshake bus interfaces (Appendix A.9).
- We show that the framework can produce formally verified hardware, including machine-checked equivalence proofs between specifications and optimized implementations, bridging the gap between LLM-generated code and provably correct hardware.

## 2 Related Work

**Hardware Abstractions and Verification.** The limitations of Verilog and VHDL motivate a rich line of work on higher-level HDLs and formal methods. Tools such as Chisel [Bachrach et al., 2012], Bluespec [Nikhil, 2004], and Clash [Baaij et al., 2010] raise the level of abstraction and eliminate classes of bugs by construction, while High-Level Synthesis (HLS) [Cong et al., 2011, Canis et al., 2011] compiles C/C++ to RTL at the cost of micro-architectural control. However, these tools often still rely on post-hoc formal verification via model checking [Clarke et al., 2018, Biere, 2009] or equivalence checking [Brayton and Mishchenko, 2010] after the RTL is generated. An alternative approach embeds hardware directly in proof assistants to enable *verification by construction*, such as Kami [Choi et al., 2017] and Silver Oak [Project Oak, 2022] in Coq, or ACL2 for industrial verification [Hunt et al., 2017]. CKTFORMALIZER builds on this tradition by targeting Lean [Moura and Ullrich, 2021], leveraging its dependent types and metaprogramming to provide both seamless SystemVerilog extraction and rich compile-time feedback that uniquely benefits LLM agents.

**LLMs for Hardware Design.** The application of LLMs to hardware design has gained momentum. Early work explored generating Verilog from English descriptions [Pearce et al., 2020], and recent general-purpose code models [Chen et al., 2021, Rozière et al., 2023, Lozhkov et al., 2024] and fine-tuned models such as VeriGen [Thakur et al., 2024] have been evaluated on Verilog tasks using

Table 1: Comparison of the traditional EDA flow and the CKTFORMALIZER flow.

| Capability                | Trad.    | Ours        |
|---------------------------|----------|-------------|
| Target language           | Verilog  | Lean DSL    |
| Compile-time width safety | ✗        | ✓           |
| Comb. loop prevention     | ✗        | ✓           |
| Exhaustive case coverage  | ✗        | ✓           |
| Feedback latency          | Hours    | ms          |
| Error diagnostics         | EDA logs | Type errors |
| Formal equiv. proofs      | ✗        | ✓           |
| Auto Verilog extraction   | N/A      | ✓           |

```

-- Example 1: 8-bit counter (sequential circuit)
def counter {dom : DomainConfig} : Signal dom (BitVec 8) :=
  Signal.circuit do
    let count <- Signal.reg 0#8
    count <~ count + 1#8
    return count

-- Example 2: Bit-width mismatch caught at compile time
def bad_add (a : Signal dom (BitVec 8))
  (b : Signal dom (BitVec 16)) : Signal dom (BitVec 8) :=
  a + b -- ERROR: type mismatch (BitVec 8 vs BitVec 16)

-- Example 3: Combinational loops impossible by construction
-- Self-referencing signals are rejected (no implicit feedback path):
def bad_loop (x : Signal dom (BitVec 8)) : Signal dom (BitVec 8) :=
  let y := x + y -- ERROR: unknown identifier 'y'
  y

-- Example 4: Exhaustive pattern matching prevents unintended latches
def mux3 (sel : Signal dom (BitVec 2))
  (a b c : Signal dom (BitVec 8)) : Signal dom (BitVec 8) :=
  Signal.match sel fun
  | 0b00 => a
  | 0b01 => b
  | 0b10 => c
  | _    => 0#8 -- must cover all cases; omitting causes compile error

```

Figure 1: Lean HDL examples illustrating key structural guarantees. (1) An 8-bit counter: `Signal.circuit` desugars into `Signal.loop` and `Signal.register`, compiling to `always_ff`. (2) Bit-width mismatches are compile-time type errors. (3) Combinational loops are impossible by construction; self-referencing signals are rejected as unknown identifiers since feedback requires explicit `Signal.reg`. (4) Exhaustive pattern matching prevents unintended latches; omitting any case is a compile error, unlike Verilog’s `always` blocks.

benchmarks such as VerilogEval [Liu et al., 2023] and RTLLM [Lu et al., 2024]. Conversational frameworks such as Chip-Chat [Blocklove et al., 2023] and ChipGPT [Chang et al., 2024] further explore multi-step hardware design flows. A common limitation across this body of work is that LLMs generate Verilog directly, inheriting all of its pitfalls: the generated code may contain width mismatches, unintended latches, or combinational loops that pass syntax checks but fail in synthesis or silicon. Our work addresses this by redirecting the LLM to target a type-safe HDL, where the compiler catches these errors immediately and provides structured feedback for iterative correction.

**LLM Agents and Automated Theorem Proving.** Beyond single-pass generation, tool-augmented LLM agents show strong results in software engineering (e.g., ReAct [Yao et al., 2023], SWE-agent [Yang et al., 2024a]). These agents can resolve real-world issues by reading code, editing files, and running tests in a feedback loop [Jimenez et al., 2023]. Concurrently, LLMs are applied to formal theorem proving, with systems such as GPT-f [Polu and Sutskever, 2020] and LeanDojo [Yang et al., 2024b] demonstrating the ability to generate proof steps in Lean. Our framework synthesizes these advances: the coding agent follows an iterative generate-compile-repair paradigm, but operates in the hardware domain where the Lean provides substantially richer feedback than typical software build errors. Furthermore, because CKTFORMALIZER operates inside Lean, the agent can generate not only correct-by-construction hardware implementations but also formal equivalence proofs that guarantee functional equivalence between specifications and optimized implementations, bridging agentic code generation and machine-checked verification.

### 3 Background: Hardware in Lean

Lean HDL is a domain-specific language embedded in Lean that describes hardware using `Signal` combinators. A signal `Signal dom  $\alpha$`  represents a synchronous stream of values of type  $\alpha$  under clock domain `dom`. Semantically, a signal is a function from discrete time steps to values: `Signal dom  $\alpha \triangleq \mathbb{N} \rightarrow \alpha$` . This denotational semantics enables both cycle-accurate simulation (by evaluating the function) and formal reasoning (by proving properties over the function). The embedding in Lean provides four properties that make it uniquely suited as an LLM compilation target for chip architecture.

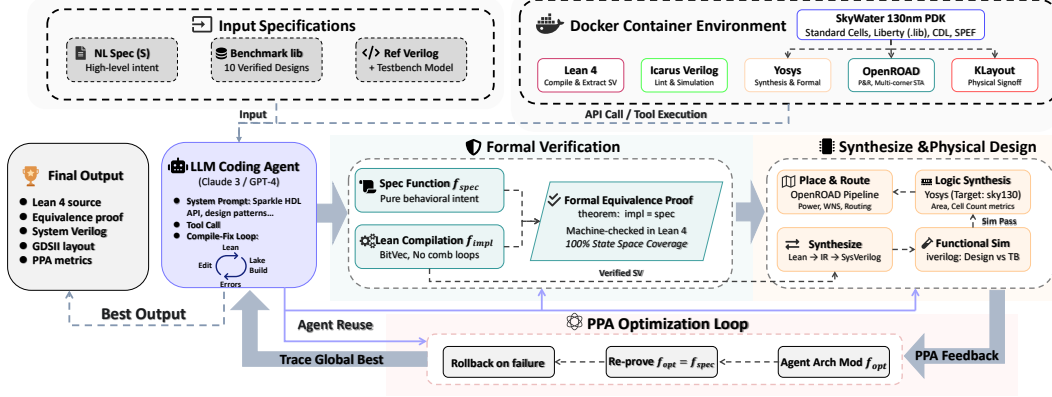


Figure 2: Overview of the CKTFORMALIZER pipeline. Given a natural language specification  $S$  and an optional reference Verilog implementation  $V_{\text{ref}}$ , a coding agent  $\mathcal{A}$  translates  $S$  into Lean code  $\mathcal{L}$  using few-shot examples and iterative error correction. The Lean compiler type-checks  $\mathcal{L}$  and extracts synthesizable SystemVerilog  $V$  via the `#synthesizeVerilog` metaprogram.  $V$  is simulated against reference testbenches, then synthesized and placed-and-routed using OpenROAD with the SkyWater 130nm PDK, yielding PPA (Power, Performance, Area) metrics. The agent receives PPA feedback and iteratively optimizes the architecture while preserving functional correctness.

**Type Safety and Structural Guarantees.** Lean’s dependent type system encodes bit widths at the type level via `BitVec N`, turning width mismatches into compile-time errors rather than silent bugs. Combinational logic forms a DAG enforced by the `Signal` monad; state feedback requires explicit register primitives (`Signal.register`, `Signal.loop`), making combinational loops impossible by construction. Lean’s exhaustive pattern matching further prevents unintended latches. Figure 1 illustrates these guarantees. For sequential circuits, CKTFORMALIZER provides an imperative `Signal.circuit` macro that desugars into `Signal.loop` and `Signal.register` calls, enabling natural descriptions of pipelines and state machines. More complex architectures (FSMs, multi-stage pipelines, memory-mapped controllers) compose from the same primitives.

**Memory, Datapath Primitives, and Formal Verification.** CKTFORMALIZER provides synthesizable memory primitives (synchronous and combinational read, pre-initialized ROM/RAM) that map to SRAM/BRAM, with address and data widths statically checked via the type signature. Combined with arithmetic operators overloaded on `Signal dom (BitVec N)`, these primitives allow complete chip architectures to be described in a single Lean file. The denotational semantics of signals further enables stating and proving correctness theorems directly in Lean (§4.3), which the PPA optimization loop (§4.2) relies on to verify that architectural transformations preserve functionality.

**Compilation and Verilog Extraction.** A central design goal is that the *compiler is the verifier*: every `lake build` invocation simultaneously type-checks the hardware description, enforces structural invariants, and extracts synthesizable RTL. Upon success, the `#synthesizeVerilog` metaprogram traverses the `Signal` combinator graph and emits clean SystemVerilog—registers as `always_ff` blocks with synchronous reset, combinational logic as `assign` or `always_comb` blocks, with clock and reset signals automatically inserted. The output passes through a Design Rule Check enforcing backend-friendly RTL conventions, and a `TopModule` wrapper is auto-generated to map Lean HDL ports to the benchmark testbench interface for drop-in simulation.

## 4 Method

We present CKTFORMALIZER, an end-to-end framework that compiles natural language hardware specifications into formally verifiable, synthesizable designs by targeting a dependently-typed HDL in Lean (Figure 2), giving the LLM agent immediate compile-time feedback for iterative refinement toward correct-by-construction hardware.

### 4.1 Coding Agent

The coding agent  $\mathcal{A}$  is an LLM equipped with tool-use capabilities that iteratively writes, compiles, and debugs Lean HDL code. We instantiate  $\mathcal{A}$  using Claude with the following components:

**Agent Configuration.** The agent is initialized with a structured system prompt encoding the HDL grammar, design patterns, common pitfalls, and a prescribed workflow. It interacts with

the environment through file system operations (read, write, edit, search) and a shell interface for compilation and simulation. Prior to generation, the agent reviews 10 verified benchmark implementations as few-shot examples. The agent operates with seven tools: a shell interface for invoking the Lean compiler and EDA tools, file read/write/edit operations for manipulating Lean source files, and search utilities (Search, Find, Browse) for locating reference implementations. Each problem is allocated a fixed turn budget, where one turn corresponds to one LLM inference followed by tool invocations.

**Iterative Compile-Fix Loop.** The agent generates an initial Lean implementation from the natural-language specification, then compiles it via the Lean toolchain. *Compilation errors (type mismatches, undefined identifiers, syntax violations) are parsed and fed back to the agent*, which applies targeted edits to the source. Because Lean HDL is embedded in Lean’s dependent type theory, type errors signal not only syntactic mistakes but also semantic violations such as width mismatches and invalid port connections. **This is the core insight of our approach:** *by using a dependently-typed HDL as the compilation target, we shift error detection from the end of the EDA pipeline (simulation, synthesis) to compile time, enabling the agent to fix semantic bugs before they propagate into costly downstream stages.* The loop repeats until compilation succeeds or the turn budget is exhausted.

## 4.2 Power, Performance, and Area Optimization Loop

For designs that pass functional simulation and synthesis, we employ a closed-loop PPA optimization strategy, a central contribution of this work. Functional equivalence between optimized variants and the original specification is guaranteed through machine-checked proofs in Lean (§4.3).

**Automated Evaluation Pipeline.** Once the agent produces a candidate Lean file, the evaluation pipeline assesses it through a sequence of increasingly stringent checks, each acting as a gate that filters out a different class of error. Type correctness and Verilog extraction are verified first: a successful build confirms that the design is semantically well-formed and produces a synthesizable .sv file. The extracted Verilog is then linted for synthesis-incompatible constructs, simulated against reference testbenches to verify functional correctness, synthesized to a gate-level netlist, and finally placed-and-routed to obtain physical PPA metrics. The pipeline is fully automated and containerized; details are in Appendix A.5.

**Architecture Exploration.** For designs with a large optimization space, the agent generates  $C$  distinct architectural candidates (default  $C=3$ ) with different micro-architectural choices (e.g., parallel vs. serial datapaths, tree vs. chain reduction). Each candidate is evaluated through the full pipeline and the best selected by PPA metrics. When formal verification is enabled, the agent additionally produces equivalence proofs between each candidate and the specification (§4.3). By coupling the LLM agent with quantitative physical design feedback, we transform hardware optimization from a manual, expert-driven process into an automated, iterative refinement loop. Each iteration proceeds through five stages: (i) extract PPA metrics from synthesis and place-and-route reports; (ii) construct a structured feedback message encoding the current metrics, optimization history, and targeted guidance; (iii) the agent modifies the Lean source to produce an optimized variant; (iv) re-evaluate the modified design through the full pipeline (compilation  $\rightarrow$  simulation  $\rightarrow$  synthesis  $\rightarrow$  P&R); and (v) accept or roll back the iteration based on an improvement criterion.

**Power, Performance, and Area Metrics.** After each synthesis and place-and-route iteration, the pipeline extracts a comprehensive PPA vector  $\mathbf{p}_i = (\text{area}_i, \text{power}_i, \text{WNS}_i, \text{cells}_i)$  from the OpenROAD reports: (i) area is measured in  $\mu\text{m}^2$  from the Yosys synthesis statistics, decomposed into sequential and combinational contributions; (ii) power is reported in  $\mu\text{W}$  from OpenROAD’s power analysis across multiple PVT corners; (iii) Worst Negative Slack (WNS) is extracted from static timing analysis across slow-slow (SS), typical-typical (TT), and fast-fast (FF) corners:  $\text{WNS} = \min_{\text{all paths}} (T_{\text{required}} - T_{\text{arrival}})$ , where  $\text{WNS} \geq 0$  indicates timing closure; and (iv) cell count provides a proxy for design complexity and routing congestion.

**Feedback Construction.** At each optimization iteration  $i$ , a structured feedback message is constructed for the agent containing: (i) the current PPA metrics  $\mathbf{p}_i$  with human-readable formatting; (ii) the complete optimization history  $\{\mathbf{p}_0, \dots, \mathbf{p}_i\}$  so the agent can identify trends; (iii) a delta analysis highlighting which metrics improved or degraded relative to the previous iteration; and (iv) targeted optimization guidance based on the dominant bottleneck (e.g., reducing intermediate signals for area-dominated designs, or pipelining long combinational paths for timing-critical designs). This structured feedback transforms raw EDA tool output into actionable intelligence that the LLM can reason about.

**Optimization Procedure.** Let  $\mathbf{p}_i$  denote the PPA vector after iteration  $i$ . The optimization proceeds for  $K$  iterations (default  $K=3$ ). At each iteration: (i) the agent resumes from its prior conversation



state with the PPA feedback appended, preserving full context of prior design decisions; (ii) it modifies the Lean source through common transformations such as replacing parallel datapaths with serialized versions, merging cascaded multiplexers into lookup tables, eliminating redundant registers, and restructuring arithmetic expressions to reduce critical path depth; (iii) the modified design is re-compiled through Lean, ensuring that the optimization did not introduce type errors, width mismatches, or structural violations—a key advantage over optimizing Verilog directly; and (iv) the design passes through the full pipeline (compilation  $\rightarrow$  simulation  $\rightarrow$  synthesis  $\rightarrow$  P&R), producing updated PPA metrics  $\mathbf{p}_{i+1}$ .

**Acceptance Criterion.** An iteration is considered an improvement if any metric improves by  $>\tau$  without any metric degrading by  $>\epsilon$ . Defining the relative change  $\Delta_m = (p'_m - p_m^*)/|p_m^*|$ :

$$\exists m : \Delta_m < -\tau \ \wedge \ \forall m : \Delta_m < \epsilon \quad (1)$$

where  $\mathbf{p}^*$  is the global best PPA vector across all iterations. This asymmetric threshold allows the agent to explore trade-offs (e.g., slightly increasing area to significantly improve timing) while preventing catastrophic regressions.

**Rollback and Termination.** If simulation fails after an optimization attempt, indicating the transformation broke functional correctness, the Lean source is rolled back to the pre-iteration version and the loop continues with the next iteration, rather than terminating early, ensuring the agent explores the full optimization budget. After all  $K$  iterations complete, the globally best design (by PPA) is restored as the final output. The entire loop is fully automated: no human intervention is required between iterations.

### 4.3 Formal Verification and Equivalence Checking

A unique capability of Lean HDL is that hardware designs can be formally verified within the same language used to describe them. Because signals have a denotational semantics as functions over time ( $\text{Signal} \text{ dom } \alpha \triangleq \mathbb{N} \rightarrow \alpha$ ), properties and equivalences are stated as standard Lean theorems and discharged by its proof kernel.

**Property and Equivalence Proofs.** Designers or the LLM agent can prove behavioral properties directly over signal definitions, such as a counter’s initial value and increment invariant (Figure 3, top). Given a specification `spec` and an optimized implementation `opt`, the agent can also prove  $\forall \text{input}. \text{opt}(\text{input}) = \text{spec}(\text{input})$  as a Lean theorem (Figure 3, bottom). This is used in the PPA optimization loop (§4.2) and architecture exploration (§4.2), where each transformation must preserve correctness. Because the proof obligation is over the *denotational semantics* of the signals, it covers all inputs and all time steps, stronger than simulation-based equivalence checking.

Table 2: Verification capabilities across three equivalence checking levels. ✓: provable, ✗: not provable, △: partially provable (depends on design complexity).

| Property                   | Lean      | Yosys    | Sim.           |
|----------------------------|-----------|----------|----------------|
| Bit-width consistency      | ✓         | —        | —              |
| No comb. loops             | ✓         | —        | —              |
| Exhaustive case coverage   | ✓         | —        | —              |
| Comb. functional equiv.    | ✓         | ✓        | △              |
| Seq. functional equiv.     | △         | △        | △              |
| Verilog extraction correct | —         | ✓        | △              |
| Multi-clock / async reset  | —         | ✗        | ✓              |
| Memory / BRAM behavior     | —         | ✗        | ✓              |
| Timing / gate-level equiv. | —         | —        | —              |
| Time complexity            | $O(p)$    | $O(2^n)$ | $O(t \cdot c)$ |
| Computational class        | Type chk. | SAT      | Linear         |

**Scope, Limitations, and Synthesis Equivalence.** Lean’s `simp` and `decide` tactics can automatically discharge equivalences for combinational circuits and simple sequential designs, but complex FSMs may require manual lemmas that the LLM agent does not always produce correctly. To complement Lean-level proofs, we employ Yosys equivalence checking, which combines SAT-based combinational equivalence reduction (`equiv_simple`) with bounded sequential induction (`equiv_induct`) to verify that the generated SystemVerilog matches the VerilogEval reference implementation at the RTL level, closing a gap that Lean-level proofs cannot address: the correctness of the Verilog extraction backend itself. Table 2 summarizes the verification capabilities across the three levels. Lean-level proofs provide the strongest guarantees for structural properties (width safety, loop freedom) and combinational equivalence, but require tactic automation that currently struggles with complex sequential logic. Yosys equivalence checking complements this by verifying RTL-level equivalence without requiring proofs, but is limited to designs where bounded induction suffices; deeply pipelined or memory-heavy designs may produce inconclusive results. Simulation provides the broadest coverage (including async resets and memories) but only over finite test vectors. Two additional categories fall outside the current verification scope. First, Lean HDL models signals under a single synchronous clock domain ( $\text{Signal} \text{ dom } \alpha \triangleq \mathbb{N} \rightarrow \alpha$ ), so multi-clock interactions and

```

-- Property proofs: counter initial value and increment invariant
theorem counter_starts_at_zero :
  (counter.atTime 0) = 0#8 := by rfl

theorem counter_increments (n : Nat) :
  (counter.atTime (n+1)) = (counter.atTime n) + 1#8 := by
  simp [counter, Signal.register]; rfl

-- Equivalence proof: optimized implementation = specification
theorem equiv (input : Signal dom (BitVec 3)) :
  optimized input = spec input := by
  unfold optimized spec; rfl

```

Figure 3: Formal verification in Lean HDL. Using the counter defined in Figure 1, the top theorems prove behavioral properties (initial value and increment invariant). The bottom theorem proves functional equivalence between an optimized implementation and its specification, providing a machine-checked correctness certificate for all possible inputs.

asynchronous resets lack a formal semantics in the current framework; verifying clock-domain crossings or async reset release timing requires either extending the denotational model to multiple time bases or relying on simulation and static CDC analysis tools. Second, gate-level and timing-aware equivalence checking operates below the RTL abstraction: post-synthesis netlist transformations, cell-level timing arcs, and physical layout parasitics are invisible to both Lean proofs and Yosys SAT checks, leaving this class of verification to sign-off tools such as gate-level simulation with back-annotated SDF timing.

## 5 Experiments

Our evaluation addresses five questions: (1) Can an LLM agent effectively generate hardware through a type-safe HDL? (2) How does this compare to direct Verilog generation? (3) Do the generated designs synthesize to physical layouts? (4) Can the PPA optimization loop improve design quality? (5) Can the agent produce machine-checked equivalence proofs between specifications and optimized implementations?

### 5.1 Benchmark

VerilogEval [Liu et al., 2023] comprises 156 HDLBits problems ranging from wire assignments to complex FSMs, each with a natural-language spec, reference Verilog, and simulation testbench. We additionally evaluate on RTLLM [Lu et al., 2024] (50 designs covering arithmetic, memory, and control modules) and ResBench [Guo and Zhao, 2025] (56 problems spanning arithmetic, signal-processing, control, and datapath designs). All three benchmarks provide reference implementations and testbenches, allowing us to assess generalization beyond any single task distribution.

### 5.2 Does a Type-Safe HDL Improve LLM-Generated Hardware?

Table 3 summarizes the end-to-end results on VerilogEval, RTLLM, and ResBench.

**Compilation Rate.** (i) *VerilogEval*. CIRCFORMALIZER achieves a 92.3% Lean compilation rate (144/156), slightly higher than the direct-SystemVerilog baseline’s 89.7% (140/156). When synthesis feedback is included in the agent loop, the rate further rises to 99.4% (155/156), indicating that many residual failures in the plain setting are repairable once synthesis constraints are exposed during iteration. (ii) *RTLLM*. CIRCFORMALIZER compiles 47/50 designs (94.0%), again above the baseline’s 46/50 (92.0%), suggesting that the type-guided generation pipeline transfers beyond HDLBits-style tasks to a more heterogeneous RTL benchmark.

**Functional Correctness.** *Designs that compile in Lean survive the backend intact—the baseline loses 20% of correct designs during synthesis and routing, whereas CIRCFORMALIZER preserves all of them.* (i) *VerilogEval*. CIRCFORMALIZER reaches 72.4% RTL simulation pass rate (113/156), matching the baseline’s 71.8% (112/156). The synthesis-guided variant attains 69.9% (109/156). (ii) *RTLLM*. RTL simulation is lower at 40.0% (20/50) vs. the baseline’s 60.0% (30/50). However, the designs that do pass are substantially more robust through the backend: CIRCFORMALIZER achieves higher gate-level simulation rates than the baseline (GLS-Synth 52.0% vs. 46.0%, GLS-P&R 50.0% vs. 48.0%) despite having fewer RTL-passing designs. *The baseline loses 20% of its functionally correct designs during synthesis and routing (GLS-P&R/Sim = 48/60), whereas CIRCFORMALIZER preserves all of them.* This gap arises because directly generated Verilog may

Table 3: End-to-end results on VerilogEval, RTLML, and ResBench. The baseline generates SystemVerilog directly in a single LLM call. RTLCoder [Liu et al., 2024] is a 7B fine-tuned model; CodeV [Zhao et al., 2025] is an instruction-tuned open-source model. CKTFORMALIZER uses iterative compile-fix with Lean type checking; “(synth)” includes Yosys synthesis in the agent loop. All methods use the same physical design flow (SkyWater 130nm HD). Sim|Comp denotes simulation pass rate conditioned on compilation. GLS-Synth and GLS-P&R denote gate-level simulation after synthesis and place-and-route. Avg. PD averages Synth, P&R, DRC, and LVS.

| Method                            | Compile       | Sim Pass     | Sim Comp     | Synth         | P&R           | DRC          | LVS           | GLS-Synth    | GLS-P&R      | Avg. PD      |
|-----------------------------------|---------------|--------------|--------------|---------------|---------------|--------------|---------------|--------------|--------------|--------------|
| <i>VerilogEval (156 problems)</i> |               |              |              |               |               |              |               |              |              |              |
| Direct SV                         | 89.7%         | 71.8%        | <b>80.0%</b> | 71.8%         | 67.9%         | 67.9%        | 67.9%         | 67.9%        | 65.4%        | 68.9%        |
| RTLCoder                          | 82.1%         | 43.6%        | 53.1%        | 43.6%         | 42.3%         | 42.3%        | 42.3%         | 43.6%        | 42.3%        | 42.6%        |
| CodeV                             | 91.7%         | 42.3%        | 46.2%        | 41.7%         | 40.4%         | 40.4%        | 40.4%         | 41.7%        | 40.4%        | 40.7%        |
| CKTFORMALIZER                     | 92.3%         | <b>72.4%</b> | 78.5%        | 91.0%         | 91.0%         | 91.0%        | 91.0%         | <b>71.2%</b> | <b>71.8%</b> | 91.0%        |
| CKTFORMALIZER (synth)             | <b>99.4%</b>  | 69.9%        | 70.3%        | <b>99.4%</b>  | <b>95.5%</b>  | <b>95.5%</b> | <b>95.5%</b>  | 68.6%        | 66.7%        | <b>96.5%</b> |
| <i>RTLML (50 problems)</i>        |               |              |              |               |               |              |               |              |              |              |
| Direct SV                         | 92.0%         | <b>60.0%</b> | <b>65.2%</b> | 56.0%         | 56.0%         | 56.0%        | 56.0%         | 46.0%        | 48.0%        | 56.0%        |
| RTLCoder                          | 70.0%         | 22.0%        | 31.4%        | 20.0%         | 20.0%         | 20.0%        | 20.0%         | 18.0%        | 18.0%        | 20.0%        |
| CodeV                             | 80.0%         | 32.0%        | 40.0%        | 32.0%         | 30.0%         | 30.0%        | 30.0%         | 26.0%        | 26.0%        | 30.5%        |
| CKTFORMALIZER                     | 94.0%         | 40.0%        | 42.6%        | 94.0%         | 90.0%         | 90.0%        | 90.0%         | <b>52.0%</b> | 50.0%        | 91.0%        |
| CKTFORMALIZER (synth)             | <b>96.0%</b>  | 54.0%        | 56.3%        | <b>96.0%</b>  | <b>96.0%</b>  | <b>96.0%</b> | <b>96.0%</b>  | <b>52.0%</b> | <b>52.0%</b> | <b>96.0%</b> |
| <i>ResBench (56 problems)</i>     |               |              |              |               |               |              |               |              |              |              |
| Direct SV                         | 98.2%         | <b>64.3%</b> | 65.5%        | 58.9%         | 58.9%         | 58.9%        | 58.9%         | 57.1%        | 57.1%        | 58.9%        |
| RTLCoder                          | 85.7%         | 51.8%        | 60.4%        | 48.2%         | 48.2%         | 46.4%        | 48.2%         | 48.2%        | 48.2%        | 47.8%        |
| CodeV                             | 100.0%        | 50.0%        | 50.0%        | 46.4%         | 46.4%         | 46.4%        | 46.4%         | 46.4%        | 46.4%        | 46.4%        |
| CKTFORMALIZER                     | 89.3%         | 60.7%        | <b>68.0%</b> | 87.5%         | 87.5%         | 85.7%        | 87.5%         | <b>58.9%</b> | <b>58.9%</b> | 87.1%        |
| CKTFORMALIZER (synth)             | <b>100.0%</b> | 60.7%        | 60.7%        | <b>100.0%</b> | <b>100.0%</b> | <b>98.2%</b> | <b>100.0%</b> | <b>58.9%</b> | <b>60.7%</b> | <b>99.6%</b> |

contain constructs that are functionally correct under simulation but fragile under logic optimization and physical implementation, such as implicit latches from incomplete sensitivity lists, redundant logic that synthesis tools remove non-equivalently, or timing-sensitive constructs that break under gate-level delays. *Designs that compile in Lean are free of width mismatches, combinational loops, and incomplete case analysis by construction*, making them inherently more resilient to backend transformations. Despite comparable simulation pass rates (71.8% vs. 72.4%), *the type-safe HDL trades marginal pass-rate parity for substantially stronger structural guarantees*: 98.6% of compiled designs produce physical layouts through the full synthesis and P&R flow, a gap we expect to narrow as LLMs gain exposure to Lean hardware descriptions. Appendix A.7 gives representative cases for the remaining simulation failures.

**Synthesis and Physical Design.** A key advantage of CIRCFORMALIZER is that compiled designs are immediately backend-ready. (i) *VerilogEval*. Plain CIRCFORMALIZER completes synthesis, place-and-route, DRC, and LVS on 142/156 designs (91.0%), compared to 112/156 synthesized and 106/156 fully routed for the direct-Verilog baseline. With synthesis feedback inside the agent loop, CIRCFORMALIZER (synth) raises these to 155/156 for synthesis and 149/156 for P&R, DRC, and LVS. (ii) *RTLML*. CIRCFORMALIZER likewise remains substantially more backend-ready, with 47/50 designs passing synthesis and 45/50 completing P&R, DRC, and LVS, versus 28/50 for all four stages in the baseline. In every setting, all designs that complete P&R also pass DRC and LVS, indicating clean layouts with no additional backend failures after routing. Gate-level simulation (Table 3, GLS columns) further confirms that these designs retain functionality through synthesis and physical implementation. *In-loop synthesis feedback is best viewed as a backend-oriented optimization*: it substantially improves deployability but does not automatically improve functional correctness, as the simulation rate slightly decreases (72.4%→69.9%).

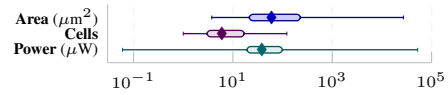
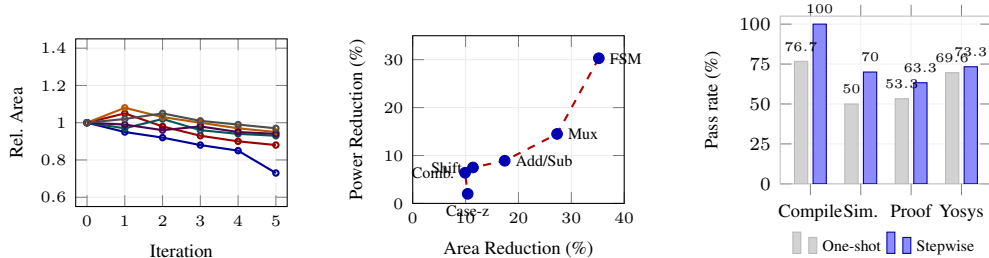


Figure 4: PPA distribution for 112 correct designs (log scale). ♦: median; shaded bars: IQR; whiskers: full range.

### 5.3 Can Type-Safe Designs Close Timing Without Manual Effort?

All designs generated by CIRCFORMALIZER that pass simulation also achieve timing closure and produce clean physical layouts, requiring no manual intervention from RTL to GDSII. Figure 4 summarizes the PPA distribution. (i) *Timing Closure*. All 113 functionally correct designs that completed place-and-route achieve WNS  $\geq 0$  ns, confirming timing-clean RTL without manual optimization. (ii) *Area and Power*. The wide range in area (3.8–24,711  $\mu\text{m}^2$ ) and power (0.06–52,500  $\mu\text{W}$ ) reflects benchmark diversity: simple combinational circuits occupy under 12  $\mu\text{m}^2$ , while





(a) PPA optimization trajectories. (b) Area-power Pareto front. (c) Proof-generation ablation.

Figure 5: (a) PPA optimization trajectories over  $K=5$  iterations (relative area, 12 designs). (b) Area-power Pareto front for architecture exploration candidates with validated simulation. (c) One-shot vs. stepwise proof generation on a 30-problem VerilogEval pilot.

complex sequential designs require thousands of  $\mu\text{m}^2$ . The median area of  $55.1 \mu\text{m}^2$  and median power of  $38.4 \mu\text{W}$  indicate most designs are compact.

#### 5.4 Can LLM Agents Discover Better Architectures via Synthesis Feedback?

We evaluate architecture exploration on 63 problems, generating  $C=4$  candidates per design with different micro-architectural choices (Table 4). (i) *Coverage*. Of 63 problems, 27 produced multiple successfully synthesized candidates, providing a meaningful basis for cross-candidate comparison. (ii) *Area and Power*. Among these 27 designs, 10 (37.0%) achieved synthesis-level area reductions exceeding 5%. For the simulation-passing candidates in Table 4, post-route power reductions range from 2.0% to 30.3%, with the largest validated area reductions coming from FSM restructuring (35.2%), a hierarchical 9-to-1 multiplexer (27.3%), and add/sub simplification (17.4%). Figure 5(b) shows the area-power Pareto front for these candidates. (iii) *Takeaway*. These results demonstrate that when an LLM agent receives concrete synthesis metrics, it can reason about hardware trade-offs at the architectural level—discovering structurally distinct implementations rather than merely tuning parameters.

Table 4: Architecture exploration: area ( $\mu\text{m}^2$ ), cell, and post-route power reductions for simulation-passing candidates.

| Design         | Type   | Init.   | Cand.   | Area         | Cell  | Power |
|----------------|--------|---------|---------|--------------|-------|-------|
| FSM (Lemmings) | Seq.   | 198.9   | 128.9   | <b>35.2%</b> | 35.0% | 30.3% |
| 9-to-1 Mux     | Comb.  | 1,818.0 | 1,321.3 | <b>27.3%</b> | 45.9% | 14.5% |
| Add/Sub Unit   | Arith. | 483.0   | 399.1   | <b>17.4%</b> | 26.2% | 8.9%  |
| FSM Shift Reg. | Seq.   | 131.4   | 116.4   | <b>11.4%</b> | 0.0%  | 7.5%  |
| Case-z Logic   | Comb.  | 83.8    | 75.1    | <b>10.4%</b> | 0.0%  | 2.0%  |
| Comb. Logic    | Comb.  | 226.5   | 203.9   | <b>9.9%</b>  | 14.8% | 6.4%  |

**PPA Optimization Loop.** We run  $K=5$  PPA optimization iterations on the 63-problem subset. Figure 5(a) shows area trajectories for the 12 designs (of 29 entering the loop) that exhibited area changes. (i) *Effectiveness*. Of 29 designs entering the optimization loop, 7 exceed the  $\tau=5\%$  area-reduction threshold (best:  $-26.8\%$  on `mux9to1v`). (ii) *Non-monotonic trajectories*. Optimization trajectories are non-monotonic—Lean’s type system rejects unsafe transformations, limiting the search space but preventing optimization-induced bugs. (iii) *Comparison with architecture exploration*. Architecture exploration (§5.4) yields larger gains by generating fresh candidates unconstrained by incremental modification. (iv) *Feedback granularity*. The optimization loop uses Yosys synthesis-level area and cell count as feedback signals; power estimation requires a full place-and-route run, which is performed only on the final design (reported in §5.3).

**Proof Generation Ablation.** *Unlike traditional SMT-solver-based equivalence checking, which is limited to bounded bit-vector reasoning, Lean proofs establish functional equivalence over arbitrary input sequences and parameterized widths.* We run a 30-problem VerilogEval pilot to isolate the effect of proof-generation strategy. Figure 5(c) compares the two modes. (i) *One-shot*. The model writes the implementation, specification, and Lean proof in a single attempt without intermediate proof-goal feedback. (ii) *Stepwise*. The model can query Lean proof states and iteratively refine the theorem. (iii) *Results*. Stepwise proof development improves all metrics, raising compile pass from 77% to 100%, simulation from 50% to 70%, proof completion from 53% to 63%, and Yosys equivalence from 70% to 73%. The consistent gains from rich intermediate compiler feedback underscore the promise of proof assistants as a foundation for hardware formal verification.

## 6 Conclusion

We presented CKTFORMALIZER, which compiles natural language hardware specifications into verifiable designs via a dependently-typed HDL in Lean. Across three benchmarks, Lean-compiled designs survive synthesis and routing intact (the baseline loses 20%), architecture exploration yields up to 35% area reduction, and the agent produces machine-checked equivalence proofs. Our results

show that dependent types and LLM agents are complementary—formal languages serve as both generation targets and verification substrates for trustworthy hardware design.

## References

- Tutu Ajayi, David Blaauw, Tuck-Boon Chan, C.-K. Cheng, Vidya A. Chhabria, D. K. Choo, M. Coltella, Ronald G. Dreslinski, Mateus Fogaca, S. Mehdi Hashemi, A. A. Ibrahim, Andrew B. Kahng, M. Kim, J. Li, Z. Liang, Uday Mallappa, Paul I. Péntzes, Geraldo Pradipta, Sherief Reda, Austin Rovinski, Kambiz Samadi, Sachin S. Sapatnekar, Lawrence K. Saul, Carl Sechen, Vaishnav Srinivas, William Swartz, Dennis Sylvester, D. Urquhart, L Wang, Mingyu Woo, and B. Xu. Openroad: Toward a self-driving, open-source digital layout implementation tool chain. 2019. URL <https://api.semanticscholar.org/CorpusID:210937106>.
- Christiaan Baaij, Matthijs Kooijman, Jan Kuper, Arjan Boeijink, and Marco Gerards. Clash: Structural descriptions of synchronous hardware using Haskell. *EUROMICRO DSD*, pages 714–721, 2010.
- Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. Chisel: Constructing hardware in a Scala embedded language. In *DAC*, pages 1212–1221, 2012.
- Armin Biere. Bounded model checking. In *Handbook of Satisfiability*, pages 457–481. IOS Press, 2009.
- Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. Chip-chat: Challenges and opportunities in conversational hardware design. *MLCAD*, pages 1–6, 2023.
- Robert Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *CAV*, pages 24–40, 2010.
- Andrew Canis, Jongsok Choi, Mark Aldham, Victor Zhang, Ahmed Kammoona, Jason H Anderson, Stephen Brown, and Tomasz Czajkowski. LegUp: High-level synthesis for FPGA-based processor/accelerator systems. In *FPGA*, pages 33–36, 2011.
- Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Li, Huawei Song, and Weikang Li. ChipGPT: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Joonwon Choi, Muralidaran Vijayaraghavan, Benjamin Sherman, Adam Chlipala, and Arvind. Kami: A platform for high-level parametric hardware specification and its modular verification. In *ICFP*, pages 1–30, 2017.
- Edmund M Clarke, William Klieber, Miloš Nováček, and Paolo Zuliani. Model checking and the state explosion problem. In *TACAS*, pages 1–30, 2018.
- Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. High-level synthesis for FPGAs: From prototyping to deployment. *IEEE TCAD*, 30(4):473–491, 2011.
- Google and SkyWater Technology. The SkyWater open source PDK. <https://github.com/google/skywater-pdk>, 2020. Accessed: 2026-05-06.
- Ce Guo and Tong Zhao. Resbench: A resource-aware benchmark for llm-generated fpga designs. In *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*, pages 25–34, 2025.
- Warren A Hunt, Matt Kaufmann, J Strother Moore, and Anna Slobodova. Industrial hardware and software verification with ACL2. In *Philosophical Transactions of the Royal Society A*, volume 375, page 20150399, 2017.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *ArXiv*, abs/2310.06770, 2023. URL <https://api.semanticscholar.org/CorpusID:263829697>.

- Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *ICCAD*, pages 1–8, 2023.
- Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. Rtlcoder: Fully open-source and efficient llm-assisted rtl code generation technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(4):1448–1461, 2024.
- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. *ASP-DAC*, pages 722–727, 2024.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. *CADE*, pages 625–635, 2021.
- Rishiyur S Nikhil. Bluespec SystemVerilog: Efficient, correct RTL from high level specifications. In *MEMOCODE*, pages 69–70. IEEE, 2004.
- Hammond Pearce, Benjamin Tan, and Ramesh Karri. Dave: Deriving automatically verilog from english. In *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, pages 27–32, 2020.
- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- Project Oak. Silver Oak: Formal specification and verification of hardware, especially for security and privacy. <https://github.com/project-oak/silveroak>, 2022. Accessed: 2026-05-07.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. Benchmarking large language models for automated Verilog RTL code generation. In *DATE*, pages 1–6, 2023.
- Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. VeriGen: A large language model for Verilog code generation. *ACM TODAES*, 29(3):1–31, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024a.
- Kaiyu Yang, Aidan M Swope, Alex Gu, Rahul Chalapathy, Peiyang Song, Shixing Yu, Maruan Al-Shedivat, Jia Lei, Pengcheng Xia, Graham Neubig, et al. LeanDojo: Theorem proving with retrieval-augmented language models. *NeurIPS*, 2024b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yinan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, et al. Codev: Empowering llms with hdl generation through multi-level summarization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.

## A Experimental Setup Details

This appendix expands the experimental setup described in Section 5.1. We include the benchmark composition, model and turn-budget settings, agent tool-use interface, and the fixed evaluation flow used to compile, simulate, synthesize, and implement generated designs.

### A.1 Benchmarks and Task Inputs

We evaluate on three RTL-generation benchmarks. VerilogEval [Liu et al., 2023] contains 156 HDLBits-derived problems spanning combinational logic, sequential circuits, counters, shift registers, and finite-state machines. RTLLM [Lu et al., 2024] contains 50 natural-language RTL design tasks covering arithmetic, memory, control, and miscellaneous modules. ResBench [Guo and Zhao, 2025] contains 56 RTL tasks with arithmetic, signal-processing, control, and application-level datapath designs. Each benchmark task provides a design description and an evaluation testbench; tasks also include reference RTL used by the harness for benchmark packaging, comparison, or wrapper construction when required.

For the CIRCFORMALIZER runs, the agent generates a Lean HDL implementation for each target problem and writes it to a task-specific generated source file. The same benchmark testbench is then used to evaluate the SystemVerilog extracted from Lean. The direct-SystemVerilog baseline uses the same backbone model but generates SystemVerilog directly in a single pass, isolating the effect of the Lean HDL target and iterative compile-fix feedback.

### A.2 Model and Hyperparameters

All LLM-based experiments use Claude Sonnet 4.5 (claude-sonnet-4-5) with temperature 0. Unless otherwise specified, each problem is allocated a generation budget of  $T=80$  turns. A turn corresponds to one LLM inference followed by zero or more tool invocations; therefore, the tool interface is part of the experimental setup because it defines the agent’s action space and the feedback channels available during generation.

**System Prompt.** The agent receives a structured system prompt (~345 lines) organized into seven sections summarized in Table 5. When architecture exploration or PPA optimization feedback is enabled, the prompt is extended with architectural dimensions to explore (parallelism, pipeline depth, data flow, resource sharing), Lean HDL patterns for each variant, and instructions for writing verified equivalence proofs using Lean tactics.

**Few-Shot Examples.** Before generating a target design, the agent can inspect a library of 10 verified Lean HDL benchmark implementations located in `Benchmark/*.lean`. These examples cover the major design patterns (combinational, sequential, FSM) and serve as in-context demonstrations of correct API usage and idiomatic Lean HDL style.

**Feedback Prompts.** Four structured feedback templates guide the agent during iterative refinement:

1. *Synthesis feedback* — reports the current pipeline status (compile, SV extraction, lint, simulation, synthesis) together with synthesis error context (tailed from Yosys/ORFS logs, up to 2,500 characters), and instructs the agent to edit the Lean source while preserving functional behavior.
2. *PPA optimization feedback* — presents the current PPA vector (area, cell count, WNS, power), a history table of all prior iterations, and an optimization focus (timing closure if  $WNS < 0$ , area reduction otherwise). The agent is instructed to define both a `_spec` (original) and an optimized variant, prove equivalence via Lean tactics (`unfold`, `ext`, `simp`, `bv_omega`), and only then commit the optimization.
3. *Architecture exploration feedback* — tabulates all candidates (description, simulation pass, verified status, area, cells, WNS, power) and optional area/latency constraints, then suggests architectural dimensions to explore (parallelism, pipelining, resource sharing).
4. *Direct SystemVerilog feedback* — used when optimizing exported Verilog rather than Lean source; includes the current SystemVerilog code, PPA metrics, and history, with instructions to preserve the module interface and output a complete SystemVerilog-2012 module.



Table 5: Structure of the CIRCFORMALIZER agent system prompt (~345 lines).

| Section                             | Content                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role & Goal                         | Expert hardware engineer role. Goal: produce Lean HDL that (1) compiles with <code>lake build</code> , (2) generates SystemVerilog via <code>#synthesizeVerilog</code> , (3) passes functional simulation.                                                                                                                                                                                                                                                                                                                            |
| Tool Interface                      | Seven tools: <code>bash</code> (shell, 120s timeout), <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>grep</code> , <code>glob</code> , <code>list_directory</code> .                                                                                                                                                                                                                                                                                                                               |
| File Template                       | Mandatory structure: <code>import CktFormalizer / import CktFormalizer.Compiler.Elab, open CktFormalizer.Core.{Domain, Signal}, function definition with {dom : DomainConfig} implicit, #synthesizeVerilog invocation. Naming: prob&lt;NNN&gt;_&lt;name&gt;.</code>                                                                                                                                                                                                                                                                   |
| Type System & API                   | Types: <code>Signal dom (BitVec N)</code> , <code>Signal dom Bool</code> , product types via <code>bundle2</code> . Operators: <code>~~~</code> , <code>&amp;&amp;&amp;</code> , <code>   </code> , <code>^^^</code> , <code>&gt;&gt;</code> , <code>&lt;&lt;</code> , <code>==</code> , <code>+</code> , <code>-</code> . API: <code>Signal.mux</code> , <code>Signal.register</code> , <code>Signal.loop</code> , <code>Signal.pure</code> , <code>Signal.map</code> , <code>hw_cond</code> . Constants: <code>N#W</code> literals. |
| Design Patterns                     | Five patterns with complete code examples: (1) combinational, (2) sequential feed-forward (register chain), (3) sequential with feedback (counter/FSM via <code>Signal.loop</code> ), (4) multi-output ( <code>bundle2</code> ), (5) FSM with explicit state encoding ( <code>private abbrev</code> ).                                                                                                                                                                                                                                |
| Critical Rules                      | Nine rules: no <code>if-then-else</code> (use <code>Signal.mux</code> ); implicit clock (no <code>clk</code> input); <code>Signal.loop</code> must end with <code>Signal.register</code> ; <code>Signal.mux</code> true-branch first; multi-output via product types; state abbreviations as <code>private abbrev</code> ; use <code>Signal.map</code> for bit extraction; exact bit-width matching; use <code>Signal.circuit</code> with <code>&lt;~</code> for imperative style.                                                    |
| Workflow & Errors                   | Six-step workflow: (1) read <code>Benchmark/*.lean</code> examples, (2) analyze problem type, (3) write <code>Generated/&lt;prob_id&gt;.lean</code> , (4) <code>lake build</code> to compile, (5) fix errors iteratively via <code>edit_file</code> , (6) verify generated SystemVerilog. Common error patterns: type mismatch, unknown identifier, <code>Signal.loop</code> issues, missing <code>open</code> statements, <code>if-then-else</code> usage.                                                                           |
| Architecture Exploration (optional) | Architectural dimensions: parallelism (fully parallel / partial / sequential), pipeline depth, data flow (systolic / broadcast / streaming), resource sharing (dedicated / time-multiplexed). Lean HDL patterns for each variant. Verified exploration: define <code>_spec</code> + optimized variant, prove equivalence via <code>unfold/ext/simp/bv_omega</code> , fall back to <code>sorry</code> if proof fails.                                                                                                                  |

The concrete prompt templates are instantiated with the current problem ID, evaluator status, and EDA log excerpts. The boxes below show the exact message structure used by the agent, with runtime fields shown in angle brackets.

## Feedback Prompt 1: Generation-time synthesis repair

```
## Generation-Time Synthesis Feedback -- Iteration <i>

Your current 'Generated/<prob_id>.lean' was generated and evaluated.
Use the feedback below to repair the Lean HDL implementation so it remains
functionally correct and becomes synthesizable.
Do not do PPA optimization here; only fix synthesizability or any regression
introduced by the previous repair.

### Current status
- compile_pass: <true/false>
- sv_extracted: <true/false>
- lint_pass: <true/false>
- sim_status: <sim_pass/sim_fail/sim_error>
- synth_pass: <true/false>

### Diagnostics
```text
<tailed Yosys/OpenROAD synthesis diagnostics>
```

### Instructions
1. Read and edit 'Generated/<prob_id>.lean'.
2. Preserve the problem behavior and public interface.
3. Use 'lean_check' to verify Lean compilation and Verilog extraction.
4. Keep '#synthesizeVerilog' on the implementation function.
5. Write the final repaired file. The external evaluator will rerun simulation
   and synthesis.
```

## Feedback Prompt 2: Lean PPA optimization

```
## PPA Optimization Feedback -- Iteration <i>

Your implementation for '<prob_id>' passed functional simulation and synthesis.
Now optimize the design for better PPA (Power, Performance, Area).

### Current PPA Metrics
- Area: <area_um2> um^2
- Cell count: <cell_count>
- WNS (worst negative slack): <wns_ns> ns
- Power: <power_uw> uW

### PPA History
| Iter | Area (um^2) | Cells | WNS (ns) | Power (uW) |
|-----|-----|-----|-----|-----|
| baseline | <...> | <...> | <...> | <...> |
| <i> | <...> | <...> | <...> | <...> |

### Optimization Focus
- If WNS is negative, focus on breaking the critical path.
- Otherwise, focus on reducing area and cell count through logic simplification.

### Instructions (Verified Optimization)
1. Read your current 'Generated/<prob_id>.lean'.
2. Use 'lean_proof_step' to define both '<name>_spec' (original) and '<name>'
   (optimized), and note the returned environment number.
3. Use 'lean_proof_step(env=<env>)' to state
   'theorem <name>_equiv : <name> = <name>_spec := by sorry'.
4. Replace 'sorry' with tactics ('unfold', 'ext', 'simp', 'bv_omega') until no
   goals remain.
5. If equivalence cannot be proved, do not optimize; keep the original design.
6. Write the final code and verify with 'lean_check'.
7. '#synthesizeVerilog' must reference the optimized implementation.
```

### Feedback Prompt 3: Architecture exploration

```
## Architecture Exploration -- Candidate <k>

Your task: write a COMPLETELY DIFFERENT architecture for '<prob_id>'.
Do NOT tweak the existing implementation -- write a new one from scratch.

### Candidates So Far
| # | Description | Sim | Verified | Area (um^2) | Cells | WNS (ns) | Power (uW) |
|---|-----|-----|-----|-----|-----|-----|-----|
| v0 | <description> | <Pass/FAIL> | <Yes/No> | <...> | <...> | <...> | <...> |

### Constraints
- Area budget: <optional budget>
- Latency budget: <optional cycle budget>

### Suggestions
- If the initial design is combinational, try a pipelined or sequential version.
- If it uses a flat mux tree, try a hierarchical or encoded approach.
- Explore a different point in the parallelism/pipeline/resource-sharing space.

### Instructions (Verified Architecture)
Use 'lean_proof_step' to keep '<name>_spec' as the original implementation and
write '<name>' as the new architecture. Prove
'theorem <name>_equiv : <name> = <name>_spec := by ...' when possible. If the
proof cannot be completed, write the new architecture with 'sorry'; it will be
marked unverified but still evaluated by simulation.
```

### Feedback Prompt 4: Direct SystemVerilog PPA rewrite

```
## Direct SystemVerilog PPA Optimization -- Iteration <i>

Problem: '<prob_id>'
Rewrite the current SystemVerilog RTL for better PPA while preserving
functionality.

### Natural Language Spec
<problem specification>

### Current SystemVerilog
```systemverilog
<current module>
```

### Current PPA Metrics
- Area: <area_um2> um^2
- Cell count: <cell_count>
- WNS (worst negative slack): <wns_ns> ns
- Power: <power_uw> uW

### Optimization Focus
- If timing is violated, prioritize a shorter critical path and better WNS.
- Otherwise, lower area and cell count without breaking functionality.

### Output Requirements
- Keep the module name exactly '<module_name>'.
- Keep the exact external port interface.
- Output ONLY the complete rewritten SystemVerilog module.
```

For PPA optimization experiments, we use improvement threshold  $\tau=0.05$ , degradation threshold  $\epsilon=0.20$  (Section 5.1), and the optimization-iteration budget specified for the corresponding experiment. Table 6 provides a complete listing.

Table 6: Complete hyperparameter listing for CIRCFORMALIZER.

| Category                                           | Parameter                    | Value       |
|----------------------------------------------------|------------------------------|-------------|
| <i>LLM Agent</i>                                   |                              |             |
| Backbone model                                     | claude-sonnet-4-5            | —           |
| Temperature                                        | Sampling temperature         | 0           |
| Max output tokens                                  | Per-turn token budget        | 16,384      |
| Turn budget ( $T$ )                                | Turns per problem            | 80          |
| Few-shot examples                                  | Verified Lean HDL benchmarks | 10          |
| API max retries                                    | Exponential backoff retries  | 20          |
| API base delay                                     | Initial retry delay          | 30 s        |
| API max delay                                      | Maximum retry delay          | 300 s       |
| <i>Synthesis Feedback Loop</i>                     |                              |             |
| Feedback iterations                                | Max repair iterations        | 2           |
| Turns per iteration                                | Agent turns per repair       | 30          |
| <i>PPA Optimization Loop</i>                       |                              |             |
| Optimization iterations ( $K$ )                    | Iterations per design        | 3           |
| Turns per iteration                                | Agent turns per PPA iter.    | 30          |
| Improvement threshold ( $\tau$ )                   | Min. relative improvement    | 0.05        |
| Degradation threshold ( $\epsilon$ )               | Max. relative regression     | 0.20        |
| <i>Architecture Exploration</i>                    |                              |             |
| Candidates ( $C$ )                                 | Architectural variants       | 3           |
| Turns per candidate                                | Agent turns per candidate    | 40          |
| <i>Physical Design (OpenROAD + SkyWater 130nm)</i> |                              |             |
| Technology / PDK                                   | Standard-cell library        | sky130hd    |
| Clock period                                       | Target timing constraint     | 10 ns       |
| Core utilization                                   | Floorplan utilization        | 5%          |
| Placement density                                  | P&R target density           | 0.15        |
| Core aspect ratio                                  | Floorplan shape              | 1           |
| Core margin                                        | Die-to-core margin           | 2 $\mu$ m   |
| PVT corners                                        | SS / TT / FF                 | 3           |
| <i>Evaluation Timeouts</i>                         |                              |             |
| RTL simulation                                     | Icarus Verilog timeout       | 30 s        |
| Logic synthesis                                    | Yosys timeout                | 600 s       |
| Place & route                                      | OpenROAD timeout             | 900 s       |
| DRC / LVS                                          | KLayout timeout              | 300 s       |
| STA                                                | OpenSTA timeout              | 120 s       |
| Gate-level simulation                              | Post-synth/P&R sim timeout   | 120 s       |
| <i>Agent Context Limits</i>                        |                              |             |
| Bash output                                        | Max shell output             | 50,000 B    |
| File read                                          | Max file size                | 100,000 B   |
| Diagnostic context                                 | Error context window         | 9,000 chars |
| Log tail                                           | Feedback log tail            | 4,000 chars |

### A.3 Component Ablation on VerilogEval Subset

Table 7 reports a component ablation on a fixed 50-problem VerilogEval subset sampled uniformly at random without replacement using seed 20260505. The Full row is filtered from an existing backend-enabled 156-problem VerilogEval run, and the ablation rows are recomputed from per-problem result records on the same subset. All metrics use the 50 sampled problems as the denominator. Each ablation row reports the result after removing one component from the full system. Removing *Iterative Repair* disables the multi-turn repair loop after the initial Lean HDL generation, so the model cannot use compiler or evaluator feedback to edit the design. Removing *Lean REPL Feedback* keeps the outer generation loop but removes interactive Lean REPL state and proof-goal feedback, leaving the agent to rely on file edits and batch compiler diagnostics. Removing *Few-shot Examples* removes the verified Lean HDL examples from the prompt context and blocks access to the benchmark example library during generation.

Table 7: Component ablation on the fixed 50-problem VerilogEval subset. Metrics are recomputed from per-problem JSONL records and reported independently; skipped or missing downstream checks count as failures under the same 50-problem denominator.

| Removed Component  | Compile        | Sim.          | Synth          | P&R           | DRC           | LVS           | GLS-Synth     | GLS-P&R       |
|--------------------|----------------|---------------|----------------|---------------|---------------|---------------|---------------|---------------|
| None (Full)        | 50/50 (100.0%) | 35/50 (70.0%) | 50/50 (100.0%) | 47/50 (94.0%) | 47/50 (94.0%) | 47/50 (94.0%) | 34/50 (68.0%) | 32/50 (64.0%) |
| Iterative Repair   | 44/50 (88.0%)  | 22/50 (44.0%) | 32/50 (64.0%)  | 31/50 (62.0%) | 31/50 (62.0%) | 31/50 (62.0%) | 22/50 (44.0%) | 21/50 (42.0%) |
| Lean REPL Feedback | 50/50 (100.0%) | 35/50 (70.0%) | 50/50 (100.0%) | 47/50 (94.0%) | 47/50 (94.0%) | 47/50 (94.0%) | 34/50 (68.0%) | 33/50 (66.0%) |
| Few-shot Examples  | 49/50 (98.0%)  | 34/50 (68.0%) | 49/50 (98.0%)  | 47/50 (94.0%) | 47/50 (94.0%) | 47/50 (94.0%) | 34/50 (68.0%) | 32/50 (64.0%) |

#### A.4 Agent-Callable Tools

Table 8 lists the tools available to the agent-stage model. These tools are not merely implementation details: they define which repository state the model can inspect, how it can modify source files, and which compiler or simulator diagnostics can enter the feedback loop during Lean HDL development.



Table 8: Agent-callable tools available during Lean HDL generation and debugging. The two Lean REPL tools are available when the persistent Lean server initializes successfully; otherwise the agent falls back to the filesystem, search, and shell tools.

| Tool                         | Category         | Role in the Agent Loop                                                                                                                                                                                  |
|------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>bash</code>            | Shell execution  | Executes bounded shell commands from the project root. The agent uses this interface to invoke project-level checks, inspect command output, and run debugging commands subject to the harness timeout. |
| <code>read_file</code>       | File access      | Reads project files with line-oriented output, enabling the agent to inspect benchmark examples, generated Lean files, prompts, and diagnostics.                                                        |
| <code>write_file</code>      | File editing     | Creates or overwrites project files, most commonly the generated <code>Generated/{prob_id}.lean</code> implementation.                                                                                  |
| <code>edit_file</code>       | File editing     | Applies a localized edit by replacing one exact string with another, supporting surgical fixes after compiler or simulator feedback.                                                                    |
| <code>Search</code>          | Search           | Searches file contents with a regular expression, allowing the agent to find relevant API usage patterns and similar Lean HDL examples.                                                                 |
| <code>Find</code>            | Search           | Lists files matching a path pattern, such as benchmark implementations or generated design files.                                                                                                       |
| <code>Browse</code>          | Navigation       | Lists files and subdirectories so the agent can discover the local project structure.                                                                                                                   |
| <code>lean_check</code>      | Lean feedback    | Sends Lean HDL code to a persistent Lean 4 REPL process and returns errors, warnings, and generated Verilog when available. This provides a fast compile-feedback path during iterative development.    |
| <code>lean_proof_step</code> | Formal reasoning | Sends incremental proof attempts to the Lean REPL and returns proof states, goals, warnings, and environment identifiers for property or equivalence proof development.                                 |

## A.5 Evaluation Pipeline Details

The evaluation pipeline runs the same ordered checks on each candidate design. For CIRCFORMALIZER, `lake build Generated.{prob_id}` first verifies Lean type correctness and triggers Verilog extraction through the `#synthesizeVerilog` metaprogram. The build output is parsed to extract the generated `.sv` file, and a `TopModule` wrapper is generated when needed to map Lean HDL port conventions to the benchmark testbench interface. Verilator or Icarus Verilog then performs an RTL lint check to catch SystemVerilog syntax errors and synthesis-incompatible constructs before simulation. Functional correctness is evaluated with the benchmark-provided testbench using Icarus Verilog and `vvp`; the harness records whether the testbench passes and, when available, the number of output mismatches.

Designs that pass RTL simulation are sent to the physical-design flow. Yosys synthesizes each design to the SkyWater 130nm HD standard-cell library (`sky130_fd_sc_hd`) through the OpenROAD Flow Docker environment, yielding gate-level netlists and initial area estimates. The synthesized netlist then undergoes floorplanning, placement, clock-tree synthesis, routing, and finishing in OpenROAD, producing a GDSII layout. Static timing analysis reports worst negative slack (WNS) across SS/TT/FF corners, and the backend flow also records power, cell count, DRC status, and LVS status. The synthesis and P&R stages run inside a Docker container with pre-installed OpenROAD Flow and the sky130 PDK, which fixes the backend environment across all experiments.

## A.6 EDA Tools

The EDA tools in Table 9 are used by the automated implementation and evaluation flow after a candidate Lean HDL program has been generated. They compile, simulate, synthesize, implement, and sign off the generated design. When feedback is enabled, the resulting errors, pass/fail signals,

Table 9: EDA tools used by the automated evaluation and backend implementation flow.

| Tool              | Pipeline Stage                      | Role                                                                                                                                                                                                                                                                                                               |
|-------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lean 4            | Source compilation                  | Type-checks Lean HDL and runs the <code>#synthesizeVerilog</code> metaprogram to emit synthesizable SystemVerilog.                                                                                                                                                                                                 |
| Icarus Verilog    | RTL lint and simulation             | Runs <code>iverilog -t null -g2012</code> for syntax checking and <code>iverilog -g2012</code> for testbench compilation.                                                                                                                                                                                          |
| vvp               | RTL and gate-level simulation       | Executes the simulator output produced by Icarus Verilog and records testbench pass/fail status or mismatch counts.                                                                                                                                                                                                |
| Yosys             | Logic synthesis and RTL equivalence | Performs synthesis inside the OpenROAD Flow Scripts flow and is also used for RTL equivalence checks via <code>equiv_make</code> , <code>equiv_simple</code> , and <code>equiv_induct</code> .                                                                                                                     |
| OpenROAD Flow     | Backend orchestration               | A set of scripts that orchestrate the EDA backend flow (synthesis, place-and-route, DRC, LVS) inside a Docker container, enabling reproducible physical implementation from RTL to GDSII.                                                                                                                          |
| OpenROAD/OpenSTA  | Physical implementation and timing  | OpenROAD is an open-source digital design implementation tool; OpenSTA is its static timing analysis engine. Together they perform floorplanning, placement, clock-tree synthesis, routing, finishing, and static timing analysis (STA); the evaluator extracts WNS, power, and PPA metrics from OpenROAD reports. |
| KLayout           | Physical verification               | Runs DRC and LVS checks through the OpenROAD Flow signoff targets.                                                                                                                                                                                                                                                 |
| Docker            | Reproducible execution              | Runs the backend flow in the <code>openroad/orfs:latest</code> container with mounted result, log, and report directories.                                                                                                                                                                                         |
| SkyWater sky130hd | Technology target                   | Provides the <code>sky130_fd_sc_hd</code> standard-cell library and PDK configuration used for synthesis, place-and-route, timing, DRC, and LVS.                                                                                                                                                                   |
| volare            | Cell-model management               | Fetches SkyWater standard-cell simulation models when gate-level simulation requires local cell libraries.                                                                                                                                                                                                         |
| netlistsvg        | Optional visualization              | Renders Yosys-generated JSON netlists into schematic SVGs for inspection.                                                                                                                                                                                                                                          |

and PPA metrics are returned to the agent in later iterations, but the command templates and launch order are fixed by the harness.

## A.7 Error Analysis: Case Study

We manually inspect the 31 VerilogEval designs that compile in Lean but do not pass RTL simulation in the main run. The boxes below give representative cases with the archived evaluator signal and the corresponding testbench evidence.

### Case 1: Reset timing – Prob047\_dff8ar

**Prompt.** Eight D flip-flops with active-high asynchronous reset.

```
log: compile=pass, lint=pass, sim=fail, synth/pnr/gds=pass
    44 mismatches in 436 samples
tb:  reset_test(1);
    Hint: reset should be asynchronous/synchronous.
```

#### Case 2: Datapath counter logic – Prob141\_count\_clock

**Prompt.** A 12-hour BCD clock with reset priority, enable gating, carry propagation, and AM/PM rollover.

```
log: compile=pass, lint=pass, sim=fail, synth/pnr/gds=pass
     177947 mismatches in 200000 samples
tb:  Hint: reset value / reset priority
     Minute roll-over; Hour roll-over; PM roll-over
```

#### Case 3: FSM corner case – Prob155\_lemmings4

**Prompt.** A Lemmings FSM with walking, falling, digging, reset, and long-fall splatter behavior.

```
log: compile=pass, lint=pass, sim=fail, synth/pnr/gds=pass
     44 mismatches in 1003 samples
tb:  repeat(21) @(posedge clk); // splat after falling left
     repeat(24) @(posedge clk); // long-fall boundary
```

#### Case 4: Interface artifact – Prob099\_m2014\_q6c

**Prompt.** One-hot FSM next-state logic with benchmark-level port-name inconsistency.

```
log: compile=pass, lint=pass, sim=error
tb:  Prob099_m2014_q6c_test.sv:71: port 'Y2' is not a port of good1.
     Prob099_m2014_q6c_test.sv:71: port 'Y4' is not a port of good1.
     Prob099_m2014_q6c_test.sv:77: port 'Y2' is not a port of top_module1.
```

These cases show that the residual failures are not backend failures: the designs either reach physical implementation and fail the behavioral oracle, or fail at the benchmark harness interface before simulation can run.

### A.8 Example Physical Layouts

Figure 6 shows two raster renderings of OpenROAD-generated GDSII layouts for generated VerilogEval designs: Prob156 (*fancytimer*), a multi-output sequential timer with BCD display logic, and Prob097 (*mux9to1v*), a 9-to-1 multiplexer with 16-bit datapath. These examples are included as qualitative illustrations of the layouts emitted by the automated synthesis and place-and-route flow; quantitative backend pass rates are reported in Section 5.

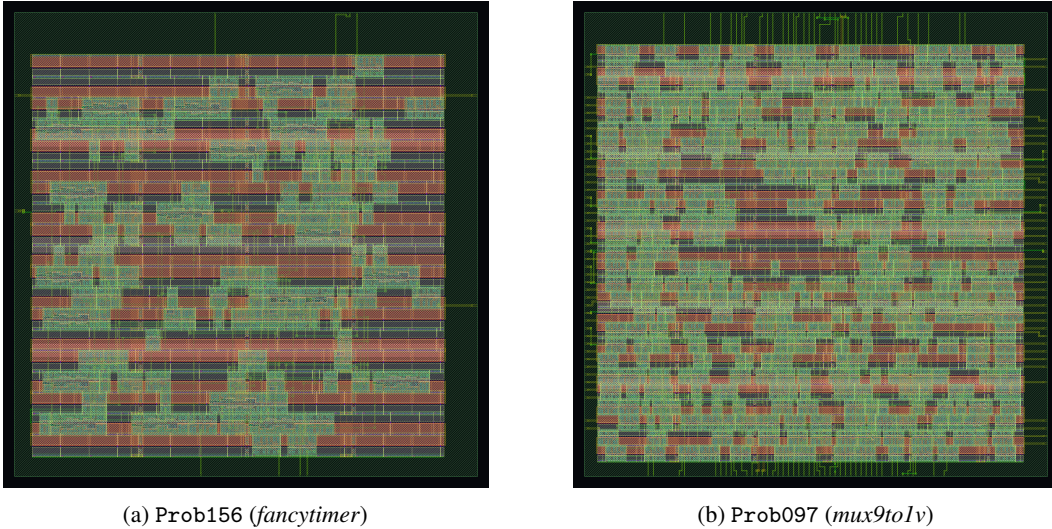


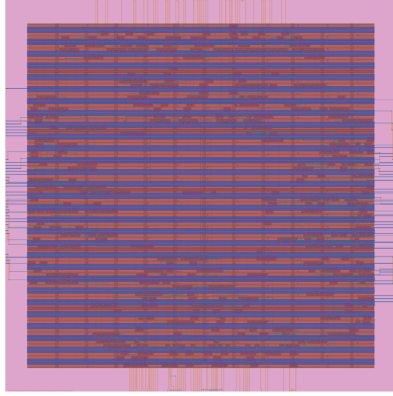
Figure 6: Raster renderings of OpenROAD-generated GDSII layouts for two VerilogEval designs.

### A.9 AXI4-Lite Physical Layouts

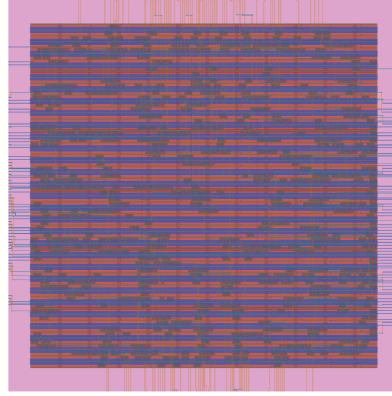
To demonstrate that CKTFORMALIZER scales beyond educational benchmarks to industry-relevant IP, we generate both sides of an AXI4-Lite bus interface, the ARM AMBA protocol used for register access in virtually all modern SoCs. AXI4-Lite requires correct multi-channel handshake logic

(AW/W/B for writes, AR/R for reads) with back-pressure and response sequencing; in hand-written Verilog, missing handshake cases and width mismatches across channels are a common source of protocol violations. In Lean HDL, the type system statically enforces channel widths and exhaustive case coverage, eliminating these error classes at compile time.

Figure 7 shows the OpenROAD-generated layouts for the target-side (slave) and initiator-side (master) modules. Both designs pass the full automated pipeline, from natural language specification through Lean compilation, SystemVerilog extraction, synthesis, and place-and-route, with zero DRC, setup, hold, slew, fanout, and capacitance violations, requiring no manual intervention at any stage.



(a) Target-side module



(b) Initiator-side module

Figure 7: OpenROAD-generated AXI4-Lite target-side and initiator-side physical layouts.



Figure 8: Six compact pass/fail dashboards.

## B Compact Result Dashboard

In addition to the aggregate metrics in Section 5, we use compact dashboards to inspect representative CKTFORMALIZER runs at problem granularity. As shown in Figure 8, the embedded view keeps only the audit signals needed for the paper: per-problem simulation outcomes and compact compile/simulation pass-rate summaries. The matrices make it easy to see whether failures are isolated to a few difficult problems or spread broadly across the benchmark.